

cflux Zeiterfassung - Konsolidierte Gesamtübersicht

Erstellt: 16. Februar 2026

Status: Konsolidierte Dokumentation aus allen Quellen

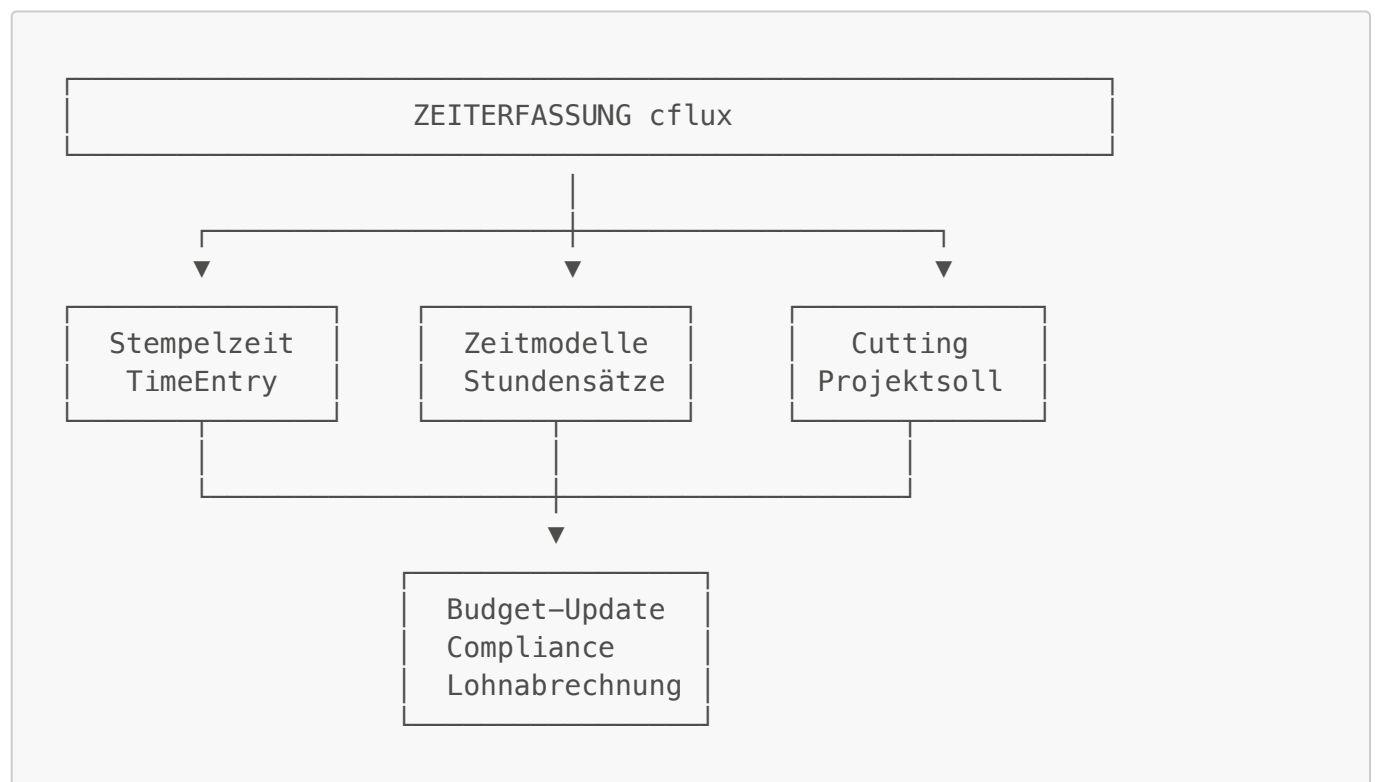
Zweck: Vollständige technische und fachliche Übersicht für Entwicklung und Copilot

Inhaltsverzeichnis

1. [Überblick & Architektur](#)
 2. [Geschäftslogik](#)
 3. [Datenmodell](#)
 4. [Zeitmodelle-System](#)
 5. [Projektsoll-Cutting](#)
 6. [Budget-Integration](#)
 7. [Compliance & Schweizer Arbeitsrecht](#)
 8. [API-Übersicht](#)
 9. [Kritische Probleme & Lösungen](#)
 10. [Implementierungs-Roadmap](#)
-

1. Überblick & Architektur

1.1 System-Komponenten



1.2 Daten-Flow



2. Geschäftslogik

2.1 Kernkonzepte

2.1.1 Stempelzeit vs. Abrechenbare Zeit

Problem (aus Meeting mit Frank):

Mitarbeiter stempelt 05:25 - 17:35 (12h 10min brutto)

Projektsoll: 06:00 - 17:00 (10h netto)

Differenz: 1h 10min ist "Mitarbeiters Bier"

Lösung:

- **Stempelzeit:** Vollständige Anwesenheit → für Lohn bezahlt
- **Projektsoll:** Kundenvorgabe → dem Kunden berechnet
- **Cutting:** Differenz wird "gecutet" (nicht abrechenbar)

```

Gestempelt:  |====05:25=====17:35====|
Projektsoll:  |====|--06:00-----17:00--|====|
Abrechenbar:  |  |*****|  |
Nicht-Abrech: |====|  |====|
                35min                35min

```

2.1.2 Zwei-Konten-System (Schweiz)

Arbeitszeitkonto (AZK):

- Enthält ALLE Überstunden über Normalarbeitszeit
- Beispiel: 50h bei 40h Normal → 10h im AZK

Überzeitkonto (ÜZK):

- **IST EIN INDIKATOR**, kein separates Konto!
- Markiert Stunden über 45h/Woche (Schweiz)
- Diese Stunden bekommen 25% Zuschlag

Abbau-Logik (LIFO - Last In First Out):

Vor Ersatzruhetag:
 AZK: 10h
 ÜZK: 5h (Indikator für Zuschlag)

Ersatzruhetag: 8h

Abbau:

1. Erst ÜZK eliminieren: 5h
2. Rest vom AZK: 3h

Nach Ersatzruhetag:
 AZK: 2h (10h – 8h)
 ÜZK: 0h (komplett abgebaut)

2.1.3 Zuschläge

Schweiz:

- Nachtarbeit (22:00-06:00): +25%
- Sonntagsarbeit: +50%
- Feiertagsarbeit: +100%
- **KEIN** Samstagszuschlag

Deutschland:

- Zusätzlich Samstagszuschlag: +50%

Wichtig: Zuschläge werden NUR auf **abrechenbare Zeit** berechnet!

2.2 Workflow-Übersicht

1. MITARBEITER STEPELT EIN
– Optionales Projekt wählen

- Optionaler Standort
- Compliance-Check: Ruhezeit (11h)
- TimeEntry: status=CLOCKED_IN



2. PAUSEN (OPTIONAL)

- Start Pause: status=ON_PAUSE
- Ende Pause: status=CLOCKED_IN
- pauseMinutes wird kumuliert



3. MITARBEITER STEMPELT AUS

- A) Compliance-Checks:
 - Tägliche Arbeitszeit (<12.5h Netto)
 - Pausen bei >6h Arbeit
 - Wöchentliche Arbeitszeit (<45h/50h)
- B) Zeitmodell-Stundensatz ermitteln:
 - Datum + Uhrzeit → passenden Stundensatz finden
 - Berücksichtigt: Wochentag, Nachtzeit, Feiertag
- C) Projektsoll-Cutting (falls konfiguriert):
 - Stempelzeit mit Projektsoll vergleichen
 - Abrechenbare Zeit berechnen
 - Nicht-abrechenbare Zeit markieren
- D) Speichern:
 - TimeEntry: status=CLOCKED_OUT
 - OvertimeBalance aktualisieren
 - Budget aktualisieren (wenn Projekt)
 - ComplianceViolation erstellen (bei Verstößen)



4. NACHBEARBEITUNG (OPTIONAL)

- Projektzeit-Verteilung:
- Stunden auf mehrere Projekte aufteilen
 - ProjectTimeAllocation erstellen
 - Validierung: Summe = Netto-Arbeitsstunden

3. Datenmodell

3.1 Hauptentitäten

```
// ===== KERN-ZEITERFASSUNG =====

model TimeEntry {
  id                String    @id @default(uuid())
  employeeId        String    // PRIMÄR (nicht userId!)
  employee           Employee  @relation(...)

  // Stempelzeiten
  clockIn           DateTime
  clockOut           DateTime?
  pauseMinutes       Int       @default(0)
  status             TimeEntryStatus // CLOCKED_IN, ON_PAUSE, CLOCKED_OUT

  // Projekt-Zuordnung
  projectId          String?
  project            Project?  @relation(...)

  // Projektsoll-Cutting (NEU)
  sollBeginn         DateTime?
  sollEnde           DateTime?
  sollPause          Int?
  abrechenbareStunden Decimal?
  nichtAbrechenbar   Decimal?
  vorSoll            Int?      // Minuten zu früh
  nachSoll           Int?      // Minuten zu spät

  // Zuschläge (NEU)
  nachtStunden       Decimal? @default(0)
  sonntagStunden     Decimal? @default(0)
  feiertagStunden    Decimal? @default(0)
  samstagStunden     Decimal? @default(0)

  // Zuschlagsberechtigungs-Flags (NEU)
  zuschlagNacht       Boolean @default(true)
  zuschlagSonntag     Boolean @default(true)
  zuschlagFeiertag    Boolean @default(true)
  zuschlagSamstag     Boolean @default(true)
  zuschlagGrund       String? // Grund wenn nicht berechtigt

  // Optionale Felder
  costCenterId       String?
  locationId         String?
  description         String?

  // DEPRECATED (wird nicht mehr verwendet!)
  userId             String? // ⚠ NICHT VERWENDEN!
}

// ===== NACHTRÄGLICHE VERTEILUNG =====

model ProjectTimeAllocation {
  id                String    @id @default(uuid())
  timeEntryId       String
```

```

    timeEntry      TimeEntry @relation(...)
    projectId      String
    project        Project    @relation(...)
    hours          Decimal    // Stunden für dieses Projekt
    description    String?
    createdAt      DateTime   @default(now())
    updatedAt      DateTime   @updatedAt
}

// ===== ÜBERSTUNDEN-SALDO =====

model OvertimeBalance {
    id              String      @id @default(uuid())
    employeeId      String
    employee        Employee    @relation(...)
    year            Int

    // Konten
    regularHours    Decimal     @default(0)    // Normalstunden
    overtimeHours   Decimal     @default(0)    // Arbeitszeitkonto (AZK)
    compensatoryHours Decimal   @default(0)    // Überzeitkonto (ÜZK) -
Indikator!

    lastUpdated     DateTime    @default(now())

    @@unique([employeeId, year])
}

// ===== COMPLIANCE-VERSTÖSSE =====

model ComplianceViolation {
    id              String      @id @default(uuid())
    employeeId      String
    employee        Employee    @relation(...)
    type            ViolationType
    date            DateTime
    actualValue      Decimal?
    allowedValue     Decimal?
    description      String?
    resolved        Boolean     @default(false)
    resolvedAt      DateTime?
    createdAt       DateTime    @default(now())
}

enum ViolationType {
    INSUFFICIENT_REST          // <11h Ruhezeit
    MAX_DAILY_HOURS            // >12.5h Tagesarbeit
    MAX_WEEKLY_HOURS           // >45h/50h Wochenarbeit
    MISSING_PAUSE              // Keine Pause bei >6h
    EXCESSIVE_CONSECUTIVE_DAYS // >6 Tage Arbeit ohne Ruhetag
}

// ===== ZEITMODELLE =====

```

```

model Zeitmodell {
  id          String    @id @default(uuid())
  name        String
  beschreibung String?
  gueltigVon   DateTime
  gueltigBis   DateTime?
  version      Int       @default(1)

  // Erweiterte Felder (NEU)
  tagesSollStunden   Decimal? @default(8.4)
  projektsollAktiv    Boolean  @default(false)
  projektsollFlexibel Boolean  @default(true)

  // Zuschlagsdefinitionen
  nachtBeginn      String? @default("22:00")
  nachtEnde        String? @default("06:00")
  nachtZuschlag    Decimal? @default(0.25)
  sonntagZuschlag  Decimal? @default(0.50)
  feiertagZuschlag Decimal? @default(1.00)
  samstagZuschlag  Decimal? @default(0.00) // CH: 0%, DE: 0.50

  eintraege      ZeitmodellEintrag[]
  zuweisungen    MitarbeiterZeitmodell[]
  aenderungen    ZeitmodellAenderung[]
}

model ZeitmodellEintrag {
  id          String    @id @default(uuid())
  zeitmodellId String
  zeitmodell   Zeitmodell @relation(...)

  stundensatz   Decimal // CHF pro Stunde
  startzeit     String   // "HH:MM"
  endzeit       String   // "HH:MM"
  wochentage    Int[]    // [0-6], leer = alle Tage
  nurFeiertage  Boolean  @default(false)
  keineFeiertage Boolean @default(false)
  prioritaeet   Int       @default(50)
}

model MitarbeiterZeitmodell {
  id          String    @id @default(uuid())
  mitarbeiterId String
  mitarbeiter  User      @relation(...)
  zeitmodellId String
  zeitmodell   Zeitmodell @relation(...)
  gueltigVon   DateTime
  gueltigBis   DateTime?

  @@unique([mitarbeiterId, zeitmodellId, gueltigVon])
}

model ZeitmodellAenderung {
  id          String    @id @default(uuid())

```

```

zeitmodellId    String
zeitmodell      Zeitmodell @relation(...)
aenderungstyp  Enum        // CREATE, UPDATE, DELETE
altJson        Json?
neuJson        Json
kommentar      String?
timestamp      DateTime @default(now())
geaendertVon   String
user           User       @relation(...)
}

// ===== PROJEKT (relevant für Cutting) =====

model Project {
  id              String      @id @default(uuid())
  name            String

  // Projektsoll-Vorgaben (NEU)
  sollBeginn      String?     // "06:00"
  sollEnde        String?     // "17:00"
  sollPauseDauer  Int?        @default(60)
  sollArbeitszeit Decimal?    // 10.0 Stunden
  cuttingAktiv    Boolean     @default(true)
  cuttingTolerance Int        @default(0) // Minuten

  // Budget
  defaultHourlyRate Decimal?
  budgetItems      ProjectBudgetItem[]

  // Relations
  timeEntries      TimeEntry[]
  allocations      ProjectTimeAllocation[]
}

```

3.2 Wichtige Indizes

```

// TimeEntry
@@index([employeeId, clockIn])
@@index([employeeId, status])
@@index([projectId, clockIn])
@@index([employeeId, datum]) // NEU für Cutting
@@index([projectId, datum]) // NEU für Cutting

// OvertimeBalance
@@index([employeeId, year])

// ComplianceViolation
@@index([employeeId, date])
@@index([type, resolved])

```


4. Zeitmodelle-System

4.1 Konzept

Zeitmodelle definieren **zeitabhängige Stundensätze** basierend auf:

- **Wochentag** (0=Montag, 6=Sonntag)
- **Uhrzeit** (z.B. 08:00-17:00, Nachtzeit)
- **Feiertage** (nur an Feiertagen, nicht an Feiertagen)
- **Priorität** (bei Überlappungen)

4.2 Stundensatz-Ermittlung

Algorithmus:

Input: mitarbeiterId, datum, uhrzeit

1. Aktives Zeitmodell für Mitarbeiter finden
(gueltigVon <= datum <= gueltigBis)
2. Wochentag bestimmen (0=Mo, 6=So)
3. Feiertag prüfen (gegen Vacation/Holiday-Tabelle)
4. Einträge filtern:
 - a) Feiertags-Filter anwenden
 - b) Wochentag prüfen (falls definiert)
 - c) Zeitbereich prüfen (startzeit <= uhrzeit < endzeit)
5. Bei mehreren Treffern:
 - Höchste Priorität gewinnt
 - Feiertags-Einträge haben implizit Vorrang

Output: Stundensatz (Decimal) oder Fehler

4.3 Mitternachts-Übergang

Einträge über Mitternacht (z.B. 22:00-06:00) werden als **zwei separate Einträge** gespeichert:

Eintrag 1: 22:00-23:59:59, Wochentage [0,1,2,3,4] (Mo-Fr Abend)
 Eintrag 2: 00:00-06:00, Wochentage [1,2,3,4,5] (Di-Sa Morgen)

4.4 Beispiel-Zeitmodell

Name: "Schichtmodell 24/7 Schweiz"

Gültig ab: 01.01.2025

Startzeit	Endzeit	Wochentage	Stundensatz	Feiertag-Regel	Priorität
-----------	---------	------------	-------------	----------------	-----------

Startzeit	Endzeit	Wochentage	Stundensatz	Feiertag-Regel	Priorität
06:00	18:00	Mo-Fr	95.00 CHF	Keine Feiertage	50
18:00	23:59:59	Mo-Fr	118.75 CHF	Alle Tage	60
00:00	06:00	Di-Sa	118.75 CHF	Alle Tage	60
00:00	23:59:59	Sa-So	142.50 CHF	Keine Feiertage	65
00:00	23:59:59	Alle	190.00 CHF	Nur Feiertage	80

Berechnungsbeispiel:

Mittwoch, 15:30 Uhr, kein Feiertag:

→ 95.00 CHF/h (Normaltarif Mo-Fr 06:00–18:00)

Mittwoch, 21:00 Uhr, kein Feiertag:

→ 118.75 CHF/h (Spättarif Mo-Fr 18:00–23:59)

Samstag, 14:00 Uhr, kein Feiertag:

→ 142.50 CHF/h (Wochenende)

1. Januar (Feiertag), 14:00 Uhr:

→ 190.00 CHF/h (Feiertag hat höchste Priorität)

4.5 Budget-Integration**Hierarchie der Stundensatz-Ermittlung:**

1. Zeitmodell-Stundensatz (höchste Priorität)
 - └ Wenn Mitarbeiter aktives Zeitmodell hat
 - └ Zeitpunkt: Clock-Out Timestamp
2. User-spezifischer Stundensatz
 - └ Fallback: User.hourlyRate
3. Projekt-Default-Stundensatz
 - └ Fallback: Project.defaultHourlyRate
4. System-Default-Stundensatz (niedrigste Priorität)
 - └ Fallback: SystemSettings.defaultHourlyRate

Budget-Update Workflow:

```

Clock-Out → hourlyRate = getHourlyRateForUser(userId, projectId, clockOut)
           → workedHours = (clockOut - clockIn - pauseMinutes) / 3600000
           → cost = workedHours × hourlyRate
           → ProjectBudgetItem aktualisieren

```

5. Projektsoll-Cutting

5.1 Konzept

Problem:

Kunde bezahlt nur für definierte Arbeitszeiten (Projektsoll).
Mitarbeiter kann außerhalb dieser Zeiten stempeln.
Differenz wird "gecutet" (nicht abrechenbar).

Lösung:

- **Stempelzeit:** Vollständige Anwesenheit (für Lohn)
- **Projektsoll:** Kundenvorgabe (für Rechnung)
- **Abrechenbare Zeit:** Überschneidung beider

5.2 Cutting-Algorithmus

```
// Pseudo-Code

function cutZeiterfassung(zeiterfassung) {
  // 1. Projektsoll ermitteln
  const soll = ermittleProjektsoll(zeiterfassung);

  // 2. Gestempelte Zeit
  const stempelBeginn = zeiterfassung.clockIn;
  const stempelEnde = zeiterfassung.clockOut;
  const pauseMinuten = zeiterfassung.pauseMinutes;

  // 3. Abrechenbare Zeit = Überschneidung
  const arbeitsBeginn = MAX(stempelBeginn, soll.beginn);
  const arbeitsEnde = MIN(stempelEnde, soll.ende);

  const abrechenbarMinuten = (arbeitsEnde - arbeitsBeginn) - soll.pause;

  // 4. Nicht-abrechenbare Zeit
  const vorSoll = MAX(0, soll.beginn - stempelBeginn);
  const nachSoll = MAX(0, stempelEnde - soll.ende);
  const nichtAbrechenbar = vorSoll + nachSoll;

  // 5. Speichern
  return {
    abrechenbareStunden: abrechenbarMinuten / 60,
    nichtAbrechenbar: nichtAbrechenbar / 60,
    vorSoll: vorSoll,
    nachSoll: nachSoll
  };
}
```

5.3 Projektsoll-Quellen

Priorität 1: Projekt-Definition

```
Project.sollBeginn = "06:00"  
Project.sollEnde = "17:00"  
Project.sollPauseDauer = 60  
→ Projektsoll: 06:00 – 17:00 mit 1h Pause
```

Priorität 2: Zeitmodell-Fallback

```
Zeitmodell.tagesSollStunden = 8.4  
→ Projektsoll: 08:00 – 17:00 mit 1h Pause (Standard)
```

5.4 Toleranz-Funktion

Wenn **cuttingTolerance** > 0:

```
Beispiel: Toleranz = 15 Minuten  
  
Mitarbeiter kommt 10 Minuten zu früh:  
→ Innerhalb Toleranz → NICHT gecuted  
  
Mitarbeiter kommt 20 Minuten zu früh:  
→ 5 Minuten werden gecuted (20min – 15min Toleranz)
```

Implementierung:

```
vorSollEffektiv = MAX(0, vorSoll – cuttingTolerance);  
nachSollEffektiv = MAX(0, nachSoll – cuttingTolerance);
```

5.5 Zuschläge auf abrechenbare Zeit

Wichtig: Zuschläge werden NUR auf die **abrechenbare Zeit** berechnet!

```
Beispiel:  
Gestempelt: 05:25 – 17:35 (davon 22:00–06:00 = Nachtzeit)  
Projektsoll: 06:00 – 17:00  
Abrechenbar: 06:00 – 17:00  
  
Nachtarbeit 05:25–06:00 (35min):  
→ NICHT abrechenbar → KEIN Nachtzuschlag!
```

Nur wenn Nachtarbeit innerhalb Projektsoll:
→ Zuschlag wird berechnet

5.6 Edge Cases

Fall 1: Komplette außerhalb Projektsoll

Projektsoll: 08:00 – 17:00
Stempel: 18:00 – 20:00 (private Nacharbeit)

Ergebnis:
abrechenbar = 0h
nichtAbrechenbar = 2h
Warnung: "Komplette außerhalb Projektsoll!"

Fall 2: Projekt ohne Sollzeit

Projekt.sollBeginn = NULL

Fallback:
→ Zeitmodell.tagesSollStunden = 8.4
→ Standard 08:00 – 17:00 mit 1h Pause
→ Hinweis: "Projektsoll nicht definiert"

Fall 3: Nachtarbeit über Mitternacht

Nachtzeit: 22:00 – 06:00
Arbeit: 23:00 – 02:00 (nächster Tag)

Berechnung:
23:00 – 00:00 = 1h Nachtzeit
00:00 – 02:00 = 2h Nachtzeit
Gesamt: 3h Nachtarbeit

6. Budget-Integration

6.1 Budget-Update Workflow

Clock-Out Event
↓
1. Stundensatz ermitteln

```

├─ Zeitmodell vorhanden?
│   └─ JA: getStundensatz(userId, clockOut)
│       └─ NEIN: User.hourlyRate → Project.defaultHourlyRate →
System.defaultHourlyRate
    |
    ▼
2. Arbeitsstunden berechnen
├─ Wenn Cutting aktiv:
│   └─ workedHours = abrechenbareStunden
├─ Sonst:
│   └─ workedHours = (clockOut - clockIn - pauseMinutes) / 3600000
    |
    ▼
3. Kosten berechnen
└─ cost = workedHours × hourlyRate
    |
    ▼
4. ProjectBudgetItem aktualisieren
├─ Category: LABOR
├─ itemName: "Max Mustermann"
├─ actualHours: 8.5
├─ hourlyRate: 125 CHF
└─ actualCost: 1062.50 CHF

```

6.2 Projekt-Zuordnung

Problem: Zwei Mechanismen für Projekt-Zuordnung

Mechanismus 1: TimeEntry.projectId

- Direktes Projekt beim Clock-In gewählt
- Einfach, schnell
- Limitierung: Nur EIN Projekt pro TimeEntry

Mechanismus 2: ProjectTimeAllocation

- Nachträgliche Verteilung auf mehrere Projekte
- Flexibel, detailliert
- Validierung: Summe muss Netto-Stunden entsprechen

Empfehlung:

Budget-Updates sollten folgende Logik verwenden:

1. Hat TimeEntry ProjectTimeAllocations?
 - └─ JA: Budget auf ALLE zugewiesenen Projekte verteilen
 - └─ NEIN: Ganzes Budget auf TimeEntry.projectId

- 2. Zeitpunkt der Budget-Verteilung:
 - └ Bei Clock-Out: TimeEntry.projectId
 - └ Bei Allokation: Umverteilen auf neue Projekte

7. Compliance & Schweizer Arbeitsrecht

7.1 ArG/ArGV 1 Anforderungen

Wichtigste Regelungen:

Regel	Grenzwert	Prüfzeitpunkt
Ruhezeit	Min. 11h zwischen Arbeitstagen	Clock-In
Tägliche Arbeitszeit	Max. 12.5h Netto	Clock-Out
Wöchentliche Arbeitszeit	Max. 45h (Büro) / 50h (Industrie)	Clock-Out
Pausen	Mind. 15min bei >5.5h, 30min bei >7h	Clock-Out
Überzeit	Über 45h → 25% Zuschlag	Wöchentlich

7.2 Compliance-Checks

Check 1: Ruhezeit (11h Minimum)

```
// Bei Clock-In
const lastClockOut = getLastTimeEntry(employeeId).clockOut;
const restHours = (now - lastClockOut) / 3600000;

if (restHours < 11) {
  createViolation({
    type: 'INSUFFICIENT_REST',
    actualValue: restHours,
    allowedValue: 11
  });
}
```

Check 2: Tägliche Arbeitszeit (Max 12.5h)

```
// Bei Clock-Out
const nettoHours = (clockOut - clockIn - pauseMinutes) / 3600000;

if (nettoHours > 12.5) {
  createViolation({
    type: 'MAX_DAILY_HOURS',
    actualValue: nettoHours,
  });
}
```

```
    allowedValue: 12.5
  });
}
```

Check 3: Wöchentliche Arbeitszeit

```
// Bei Clock-Out
const weekStart = startOfWeek(clockOut);
const weekEnd = endOfWeek(clockOut);
const weeklyEntries = getTimeEntries(employeeId, weekStart, weekEnd);

let totalHours = 0;
weeklyEntries.forEach(entry => {
  if (entry.clockOut) {
    const nettoHours = (entry.clockOut - entry.clockIn -
entry.pauseMinutes) / 3600000;
    totalHours += nettoHours;
  }
});

const limit = employee.weeklyHours || 45; // 45h oder 50h

if (totalHours > limit) {
  createViolation({
    type: 'MAX_WEEKLY_HOURS',
    actualValue: totalHours,
    allowedValue: limit
  });
}
```

Check 4: Pausen

```
// Bei Clock-Out
const bruttoHours = (clockOut - clockIn) / 3600000;
const pauseHours = pauseMinutes / 60;

let requiredPause = 0;
if (bruttoHours > 7) requiredPause = 0.5;
else if (bruttoHours > 5.5) requiredPause = 0.25;

if (pauseHours < requiredPause) {
  createViolation({
    type: 'MISSING_PAUSE',
    actualValue: pauseHours,
    allowedValue: requiredPause
  });
}
```


7.3 Überstunden-Berechnung

OvertimeBalance Update:

```
// Bei Clock-Out
const nettoHours = (clockOut - clockIn - pauseMinutes) / 3600000;
const expectedHours = employee.dailyHours || 8.4;
const overtime = nettoHours - expectedHours;

if (overtime > 0) {
  // Normale Überstunden (AZK)
  overtimeBalance.overtimeHours += overtime;

  // Überzeit-Indikator (ÜZK)
  if (weeklyHours > 45) {
    const compensatory = weeklyHours - 45;
    overtimeBalance.compensatoryHours += compensatory;
  }
}
```

Abbau-Logik (LIFO):

```
function abbauenStunden(employeeId, stunden) {
  const balance = getOvertimeBalance(employeeId);

  // 1. Erst zuschlagsberechtigte Stunden (ÜZK)
  const abzugCompensatory = Math.min(stunden, balance.compensatoryHours);
  balance.compensatoryHours -= abzugCompensatory;

  // 2. Rest vom Arbeitszeitkonto (AZK)
  const restAbzug = stunden - abzugCompensatory;
  balance.overtimeHours -= restAbzug;

  save(balance);
}
```

8. API-Übersicht

8.1 Zeiterfassung (/api/time)

Endpunkt	Methode	Auth	Funktion
/clock-in	POST	User	Einstempeln
/clock-out	POST	User	Ausstempeln + Compliance + Budget
/start-pause	POST	User	Pause beginnen

Endpunkt	Methode	Auth	Funktion
/end-pause	POST	User	Pause beenden
/current	GET	User	Aktiver Eintrag
/my-entries	GET	User	Eigene Einträge
/my-entries/:id	PUT	User	Eintrag bearbeiten
/my-entries/:id	DELETE	User	Eintrag löschen
/manual-entry	POST	Admin	Manueller Eintrag
/:id	PUT	Admin	Beliebigen Eintrag ändern
/:id	DELETE	Admin	Beliebigen Eintrag löschen
/user/:userId	GET	Admin	Einträge eines Users
/logged-in-users	GET	User	Eingestempelte Mitarbeiter

NEU für Cutting:

Endpunkt	Methode	Auth	Funktion
/:id/cutting	POST	User	Manuelles Cutting
/:id/auto-cut	POST	User	Clock-Out + Auto-Cutting
/:id/zuschlag-flags	PATCH	User	Zuschlagsberechtigungen setzen

8.2 Zeitmodelle (/api/zeitmodelle)

Endpunkt	Methode	Auth	Funktion
/	GET	Admin	Alle Zeitmodelle
/:id	GET	Admin	Einzelnes Zeitmodell
/	POST	Admin	Zeitmodell erstellen
/:id	PUT	Admin	Zeitmodell aktualisieren
/:id	DELETE	Admin	Zeitmodell löschen
/assign	POST	Admin	Mitarbeiter zuweisen
/mitarbeiter/:userId	GET	User	Zuweisungen eines Mitarbeiters
/assign/:id	DELETE	Admin	Zuweisung entfernen
/stundensatz/:userId	GET	User	Stundensatz berechnen
/abrechnung/:userId	GET	User	Arbeitszeitabrechnung

8.3 Projektzeit-Allokation (/api/project-time-allocations)

Endpoint	Methode	Auth	Funktion
/time-entry/:id	GET	User	Allokationen eines Eintrags
/time-entry/:id	POST	User	Allokationen setzen
/:id	DELETE	User	Allokation löschen
/stats	GET	User	Statistik nach Projekt

8.4 Reports (/api/reports)

Endpoint	Methode	Auth	Funktion
/my-summary	GET	User	Eigene Zusammenfassung
/user-summary/:userId	GET	Admin	User-Zusammenfassung
/all-users-summary	GET	Admin	Alle User
/project-summary/:projectId	GET	Admin	Projekt-Zusammenfassung
/project-time-by-user	GET	Admin	Projektzeit pro User
/time-bookings	GET	Admin	Detaillierte Zeitbuchungen
/user-time-bookings/:userId	GET	Admin	User-Zeitbuchungen
/overtime-report	GET	Admin	Überstunden-Report

9. Kritische Probleme & Lösungen

9.1 KRITISCH: userId vs. employeeId

Problem:

TimeEntry hat zwei Felder:

- userId (DEPRECATED, aber noch in Datenbank)
- employeeId (korrekt, primär)

Report Controller filtert FALSCH nach userId
 → Neue Einträge (nur employeeId befüllt) fehlen in Reports!

Lösung:

```
// ❌ FALSCH (alte Reports)
where: { userId }
```

```
// ✅ RICHTIG (korrigiert)
const employee = await getEmployeeId(userId);
where: { employeeId: employee.id }
```

Betroffene Dateien:

- `backend/src/controllers/report.controller.ts` (kompletter Refactor nötig)

9.2 KRITISCH: Pausenabzug fehlt**Problem:**

```
// Helper-Funktion berechnet BRUTTO-Stunden
const calculateWorkHours = (clockIn, clockOut) => {
  return (clockOut - clockIn) / 3600000; // KEINE Pausen!
};

// Verwendet in:
- getMySummary
- getUserSummary
- getAllUsersSummary
- getOvertimeReport ← BESONDERS KRITISCH!
- Compliance-Checks ← BESONDERS KRITISCH!
```

Lösung:

```
// ✅ RICHTIG
const calculateWorkHours = (clockIn, clockOut, pauseMinutes = 0) => {
  const brutto = (clockOut - clockIn) / 3600000;
  return brutto - (pauseMinutes / 60);
};
```

Betroffene Stellen:

- `report.controller.ts`: Alle Summary-Reports
- `compliance.service.ts`: `updateOvertimeBalance`, `checkWeeklyHoursViolation`, `checkDailyHoursViolation`

9.3 HOCH: Projekt-Zuordnung inkonsistent**Problem:**

Zwei Mechanismen:

1. `TimeEntry.projectId` (direkt beim Clock-In)
2. `ProjectTimeAllocation` (nachträgliche Verteilung)

Budget-Updates beachten nur (1)!

Reports teilweise nur (1), teilweise beide.

Lösung:

```
// Budget-Update-Logik
async function updateBudget(timeEntry) {
  // Prüfe ob Allokationen vorhanden
  const allocations = await getProjectTimeAllocations(timeEntry.id);

  if (allocations.length > 0) {
    // Verteilung auf ALLE Projekte
    for (const alloc of allocations) {
      updateProjectBudget({
        projectId: alloc.projectId,
        hours: alloc.hours,
        rate: hourlyRate
      });
    }
  } else {
    // Fallback: Ganzes Budget auf direktes Projekt
    updateProjectBudget({
      projectId: timeEntry.projectId,
      hours: workedHours,
      rate: hourlyRate
    });
  }
}
```

9.4 HOCH: Keine Audit-Trail bei Löschung

Problem:

```
// Admin-Löschung ohne Protokollierung
async deleteTimeEntry(id) {
  await prisma.timeEntry.delete({ where: { id } });
  // Budget nicht zurückgerechnet!
  // OvertimeBalance nicht korrigiert!
  // ComplianceViolations bleiben bestehen!
}
```

Lösung:

```
// ✅ Mit Audit-Trail und Rückrechnung
async deleteTimeEntry(id, userId) {
  const entry = await prisma.timeEntry.findUnique({ where: { id } });

  // 1. Budget zurückrechnen
  await revertBudgetUpdate(entry);

  // 2. OvertimeBalance korrigieren
  await revertOvertimeUpdate(entry);
}
```

```
// 3. ComplianceViolations entfernen/markieren
await removeRelatedViolations(entry);

// 4. Soft-Delete oder Hard-Delete mit Logging
await prisma.timeEntry.update({
  where: { id },
  data: { deleted: true, deletedBy: userId, deletedAt: new Date() }
});

// 5. Action-Log
await actionService.logAction({
  type: 'TIME_ENTRY_DELETED',
  userId,
  data: entry
});
}
```

10. Implementierungs-Roadmap

Phase 1: Kritische Fixes (SOFORT)

Priorität: HÖCHSTE

- ☐ Report Controller: userId → employeeId Migration
 - └ Alle WHERE-Klauseln korrigieren
 - └ Tests durchführen
- ☐ Pausenabzug in calculateWorkHours
 - └ Helper-Funktion erweitern
 - └ Alle Aufrufe anpassen
- ☐ Compliance-Service: Pausen berücksichtigen
 - └ checkWeeklyHoursViolation
 - └ checkDailyHoursViolation
 - └ updateOvertimeBalance

Dauer: 2-3 Tage

Risiko: HOCH wenn nicht gemacht

Phase 2: Zeitmodelle (KW 8-9)

Priorität: HOCH (für Budget-Korrektheit)

- ☐ Prisma Schema erweitern
 - └ Zeitmodell-Felder
 - └ Migration erstellen
- ☐ ZeitmodellService implementieren

- └─ getStundensatz()
- └─ berechneAbrechnung()
- Budget-Integration
 - └─ getHourlyRateForUser() erweitern
 - └─ Zeitmodell-Lookup einbauen
- Admin-UI für Zeitmodelle
 - └─ Zeitmodell-Verwaltung
 - └─ Mitarbeiter-Zuweisung

Dauer: 1 Woche

Dependencies: Keine

Phase 3: Projektsoll-Cutting (KW 10-11)

Priorität: HOCH (für Kundenabrechnung)

- Prisma Schema erweitern
 - └─ TimeEntry: Cutting-Felder
 - └─ Project: Projektsoll-Felder
- CuttingService implementieren
 - └─ cutZeiterfassung()
 - └─ ermittleProjektsoll()
 - └─ berechneAbrechenbar()
 - └─ berechneZuschlage()
- API Endpoints
 - └─ POST /time/:id/cutting
 - └─ POST /time/:id/auto-cut
 - └─ PATCH /time/:id/zuschlag-flags
- Frontend-Komponenten
 - └─ ZeiterfassungCuttingAnzeige
 - └─ ZuschlagFlagEditor
 - └─ ProjektsollKonfigurator

Dauer: 1-2 Wochen

Dependencies: Zeitmodelle (für Zuschläge)

Rollout: Anfang März (Testgruppe)

Phase 4: Projekt-Zuordnung vereinheitlichen (KW 12)

Priorität: MITTEL

- Budget-Update erweitern
 - └─ ProjectTimeAllocation berücksichtigen
 - └─ Verteilung auf mehrere Projekte

- Reports korrigieren
 - └─ getProjectSummary: Allokationen einbeziehen
 - └─ getProjectTimeByUser: Allokationen einbeziehen
- Validierung verbessern
 - └─ Summe der Allokationen = Netto-Stunden

Dauer: 3–4 Tage

Dependencies: Phase 1 (kritische Fixes)

Phase 5: Audit & Qualität (KW 13+)

Priorität: NIEDRIG (aber wichtig)

- Audit-Trail für Löschungen
 - └─ Soft-Delete Pattern
 - └─ Budget-Rückrechnung
 - └─ OvertimeBalance-Korrektur
- Überlappungs-Validierung
 - └─ Prüfung bei manuellen Einträgen
 - └─ UI-Warnung bei Konflikten
- Performance-Optimierung
 - └─ Prisma Client Singleton
 - └─ Report-Queries optimieren
- Testing
 - └─ Unit Tests für Cutting
 - └─ Unit Tests für Zeitmodelle
 - └─ Integration Tests

Dauer: Laufend

Dependencies: Alle vorherigen Phasen

Anhang A: Glossar

Begriff	Bedeutung
AZK	Arbeitszeitkonto - enthält alle Überstunden
ÜZK	Überzeitkonto - Indikator für zuschlagsberechtigte Stunden (>45h)
Cutting	Abschneiden nicht-abrechenbarer Zeiten außerhalb Projektsoll
Projektsoll	Vom Kunden vorgegebene Arbeitszeiten
Abrechenbare Zeit	Zeit innerhalb Projektsoll, die dem Kunden berechnet wird

Begriff	Bedeutung
Netto-Zeit	Arbeitszeit minus Pausen
Brutto-Zeit	Gesamte Anwesenheit (inkl. Pausen)
LIFO	Last In First Out - Abbau-Strategie (zuletzt erworbene Stunden zuerst)
ArG	Arbeitsgesetz (Schweiz)
ArGV 1	Verordnung 1 zum Arbeitsgesetz (Schweiz)

Anhang B: Wichtige Dateien

Datei	Pfad	Verantwortlichkeit
Schema	<code>backend/prisma/schema.prisma</code>	Datenmodell
Time Controller	<code>backend/src/controllers/time.controller.ts</code>	Zeiterfassung
Report Controller	<code>backend/src/controllers/report.controller.ts</code>	Reports (⚠ KRITISCH)
Compliance Service	<code>backend/src/services/compliance.service.ts</code>	ArG-Checks (⚠ KRITISCH)
Budget Service	<code>backend/src/services/budgetUpdate.service.ts</code>	Budget-Updates
Zeitmodell Service	<code>backend/src/services/zeitmodell.service.ts</code>	Stundensatz-Ermittlung
Cutting Service	<code>backend/src/services/cutting.service.ts</code>	Projektsoll-Cutting (NEU)
Hourly Rate Service	<code>backend/src/services/hourlyRate.service.ts</code>	Stundensatz-Hierarchie

Ende der Dokumentation

Nächste Schritte:

1. Kritische Fixes durchführen (Phase 1)
2. Zeitmodelle implementieren (Phase 2)
3. Projektsoll-Cutting entwickeln (Phase 3)
4. Testing und Rollout (Anfang März)